

3 Stones

The Objective

3 Stones is a board game for two players. The players score points by placing their stones in pockets.

The Players

Two players play this game - one player will be the human user of your program, and the other player will be your program/computer. The two players will play a "tournament", consisting of one or more "rounds". Each round will consist of the two players taking "turns" till one of them wins.

The Setup

The board: The game is played on an octagonal board consisting of 80 pockets laid out in 11 rows and 11 columns, as shown below, where pockets are represented by O:

```

      0 0 0
    0 0 0 0 0
  0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0
    0 0 0 0 0 0 0
      0 0 0

```

Note that the center row has a void in the center.

Stones:: Each player starts with 15 black stones, 15 white stones and 6 clear stones.

Score card: Each player maintains a score card, initially initialized to zero.

A Round

The two players take alternate turns till one of them wins.

A Turn

During one's turn, a player places one of their stones in an empty pocket on the board.

- Valid placement:
 - If this is the first turn of the round, the player can place it anywhere on the board.
 - Otherwise, the player must place the stone in an empty pocket either in the same row or in the same column as the opponent's last turn (but not necessarily adjacent to the stone last played by the opponent).
 - If no pocket is available on the row and column of the opponent's last play, the stone can be

placed in any empty pocket on the board.

- Objective: The objective of each turn is to earn points:
 - The player earns one point by placing 3 stones in adjacent pockets:
 - The stones may be in a row, column or diagonal
 - The stones must all be of the player's color or clear color
 - Clear stones count for either player. As a result, when a clear stone (C) is placed by either player, both players may get a point each, as in the following scenarios:

```

      B
    W C W
      B
  
```

```

W W C B B
  
```

```

      B
      B
    W W C
  
```

- But, a row of 3 clear stones will not count for either player
 - A stone may be part of more than one scoring arrangement, e.g., each of the following arrangements of white stones earns 2 points:

```

W W W W
  
```

```

      W
    W W W
      W
  
```

```

      W
    W W W
      W
  
```

```

    W   W
      W
    W   W
  
```

```

    W
    W
    W
    W
    W
  
```

- The newly placed stone **must** be part of any 3 stone-sequence for which points are earned. For example, consider the following, where asterisk shows the pocket in which a new W stone was placed:

```

WW*WWWW
  
```

This will earn the player 3 points:

```

WW*  W*W  *WW
  
```

But, the player will **not** get points for the two sequences of three white stones that appear entirely after the asterisk.

- After each turn, the players will update their score cards. Recall that points are earned for only the *new* 3-stone arrangements created in that turn for either player.

End of Round

The round ends when the last stone is played.

Winner

When the round ends, the player with the most points wins. If both the players have the same number of points, the round is a draw.

First Player

Initially, a coin is tossed and the human player is asked to call head or tail.

- If the human player calls the coin toss correctly, the human player is asked to pick a color: white or black. The computer takes the other color.
- If the human player calls the coin toss incorrectly, computer picks the color and human player takes the

other color.

The player who picked black plays first. Thereafter, the two players alternate.

The Tournament

After each round, your program should ask the user whether they want to play another round.

- If yes, it should play another round as described above, announce the winner of the round, and repeat.
- If no, it should announce the winner of the tournament as the player who won the most rounds (*not* points), and exit.

Strategy

On every turn, the player must determine:

1. The color to play: Whether to place a stone of their color, opponent's color, or clear color
 1. Playing own color: The player may want to place their own color to:
 - A. Build towards a 3-stone arrangement in their color
 - B. Block the opponent from completing a 3-stone arrangement
 2. Playing opponent's color: The player may want to place opponen't color to block the opponent from scoring. For example, if the opponent is white and has stones arranged as:

W W

the black player may place their white stone as shown by the asterisk below:

W*W

This will not earn the black player any points, but it will prevent the white player from earning a point on the next turn by placing a white stone in the pocket represented by the asterisk. On the other hand, if the white player places a white stone on the next turn at the end of the three stones, as in:

W*WW

the white player will earn a point for

*WW

3. Playing clear stones:
 - A. The player may want to hold on to clear stones and use them only after running out of their own stones
 - B. The player may want to avoid playing a clear stone when it may also benefit the opponent, as in the examples illustrated earlier.
2. The pocket in which to place the stone.
 1. If the player has more than one pocket where they can earn points, they may want to pick the pocket that earns the most points, unless doing so also yields points to the opponent.

Implementation

- **User Interface:** You must provide a user-friendly interface for the game. For C++, LISP and Prolog, ASCII graphics and command-line input are sufficient.

- The rows of the board must be numbered, with 1 being at the top. The columns must also be numbered, with 1 being at the left margin. (For example, the leftmost pocket in the top row of the board is at row 1, column 5, i.e., (1,5))
- The pockets must be referred to by their row and column numbers, e.g., (6,5) refers to the pocket to the left of the center void.
- All human inputs must be validated.
- The turn played by the computer as well as its strategy must be displayed on the screen, e.g.,

The computer placed a white stone at (6,4).

It did so to earn a point for the three stones in a diagonal at (7,3), (6,4), (5,5)

- **Serialization:** The user should be able to suspend the game after any turn, and resume at a later time from where the game was left off. In order to do this:

- Provide the option to serialize after each turn in the game.
- When the serialization option is exercised, your program will save the current state of the game into a file and quit. We will use text format for the file.

The text format for Java/C++ will be as follows:

```
Human Player: Black 8 7 3 2 1
Computer Player: White 7 4 9 0 1
Next Player: Human 3 5
Board:
0 0 0
0 0 B 0 0
0 0 0 B W W 0
0 0 0 0 C W 0 0 0
0 0 0 0 0 B W C 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0
0 0 0
```

In the above format, human player currently has 8 points, 7 black, 3 white and 2 clear stones.

Human player has won one round so far in the tournament. Computer player currently has 7 points, 4 black, 9 white and 0 clear stones. Computer player has won one round so far in the tournament. It is the human player's turn next. The last pocket in which the opponent (in this case, computer) placed a stone before the game was suspended was on row 3 (top row is 1, bottom row is 11) and column 5 (leftmost column is 1, rightmost column is 11). Each row of the board is represented by a separate line of O (for open), W (for white), B (for black) or C (for clear).

The format for LISP is as follows. Note that entire state is just one list. The comments (preceded by semi-colon) are included here for readability - they are optional, i.e., your program does not have to save the file with comments embedded. In the sample files, comments will not be included, so your program does not have to process comments.

```
(
  ; Human Player:
  ( Black 8 7 3 2 1 )

  ; Computer Player:
  ( White 7 4 9 0 1 )

  ; Next Player:
  (Human 3 5)
```

```

; Board:
(
  ( 0 0 0 )
  ( 0 0 B 0 0 )
  ( 0 0 0 B W W 0 )
  ( 0 0 0 0 C W 0 0 0 )
  ( 0 0 0 0 0 B W C 0 0 0 )
  ( 0 0 0 0 0 0 0 0 0 0 )
  ( 0 0 0 0 0 0 0 0 0 0 )
  ( 0 0 0 0 0 0 0 0 0 )
  ( 0 0 0 0 0 0 0 )
  ( 0 0 0 0 0 )
  ( 0 0 0 )
)
)

```

The format for Prolog is as follows. Note that entire state is just one list. Lower case is used for o, w, b and c. The comments (preceded by percentage sign) are included here for readability - they are optional, i.e., your program does not have to save the file with comments embedded. In the sample files, comments will not be included, so your program does not have to process comments.

```

[
  % human Player:
  [ black, 8, 7, 3, 2, 1 ],

  % computer Player:
  [ white, 7, 4, 9, 0, 1 ],

  % Next Player:
  [ human, 3, 5 ],

  % Board:
  [
    [ o, o, o ],
    [ o, o, b, o, o ],
    [ o, o, o, b, w, w, o ],
    [ o, o, o, o, C, w, o, o, o ],
    [ o, o, o, o, o, b, w, c, o, o, o ],
    [ o, o, o, o, o, o, o, o, o, o ],
    [ o, o, o, o, o, o, o, o, o, o ],
    [ o, o, o, o, o, o, o, o, o ],
    [ o, o, o, o, o, o, o ],
    [ o, o, o, o, o ],
    [ o, o, o ]
  ]
].

```

-
- When your program is started, it will ask whether to start a fresh game or resume a game from a previously saved state.
 - If fresh game, it will start a game with a clear board and zero points for both players.
 - If resume a game, it will ask for the name of the text file from which to read the current state of the game, and resume playing from there onwards.
 - **Computer's Strategy:** The computer must play to win. First, it must generate all possible plays for the current board. Then, it must pick the "best" play.
 - It must use at least two non-trivial strategies when picking the best play, e.g., 2.1, 1.1.B, 1.2, and 1.3.B under "Strategy" section are non-trivial strategies. 1.1.A, and 1.3.A are not. You may be able

to come up with other strategies - if in doubt about whether they are non-trivial, run them by me before you implement them.

- It must "explain" the color of the stone it placed and the pocket where it placed the stone. It should list any strategy it used to place the stone.
 - **Help Mode:** When the human player is playing, the computer must provide a help mode:
 - If the human player asks for a recommendation, the computer must display:
 1. All possible plays for the human player at that point,
 2. the best next play for the human,
 3. the strategy on which the best play is based.
- The computer must use its own playing strategy to come up with this recommendation.

Assistance

You must meticulously document all the assistance you received, either from another person or from a textbook or online source including generative AI (e.g., ChatGPT). As part of your project demonstration, you will be asked to explain the workings of your code. So, it is important that you understand how any code that you borrow works. For any code that you are unable to explain, you may lose up to 30% of the grade on the project.

Grading

Please see the rubric for some of the features on which your program will be graded.

Milestones

The following are recommended milestones for each project:

- Week 1: A basic, complete game except the items listed for week 2. Attempt the game in the following order:
 1. A turn for human player
 2. A simplistic turn for computer player playing without a strategy, but within the rules of the game
 3. A game loop in which the two players take turns
 4. Deciding the first player
 5. Determining whether the round is over
 6. Determining the winner
 7. Composing all the above within a round
 8. Creating a tournament of multiple rounds and declaring the winner of the tournament
- Week 2: All the strategies for the computer player, help for human player, serialization and generalization.
- Week 3: Code refactoring, cleaning, documentation.

Acknowledgments

This game was adapted from the description at [EducationalLearningGames.com](https://www.educationallearninggames.com).